

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: Bernard G. Yasay

Program: Doctor of Engineering with Specialization in Computer Engineering

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in Loading_Dataset_and_CNN.ipynb for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	CNN activity – Manual Hyperparameter Tuning
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	Bernard G. Yasay
Date	September 27, 2026
Experiment / Trial No.	Trial number 1 out of 7 trials

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	<i>e.g. 50 / 80 / 100 / 128 / 150</i>	100	100	A bigger input image size helps to capture more image details and better represents dog/cat features. For the image size of 128×128 pixels and higher, the pixel values were normalized by X/255.0 were set to None to prevent RAM overflows and significant memory consumption. Therefore, the input size was chosen to have a good balance between the amount of captured image details and computational power.

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Color Mode	<i>Grayscale / RGB</i>	<i>Grayscale</i>	RGB	RGB preserves valuable information that can be used to differentiate fur, eyes, ears, and so on. In comparison with grayscale, RGB possesses extra color features that contribute to better classification of dogs and cats.
Pixel Normalization	<i>X/255.0 / none</i>	<i>X/255.0</i>	X/255.0	Maintains pixel values within the 0–1 range, helping the CNN achieve more stable and consistent training.
Shuffle Training Data	<i>Yes / No</i>	<i>Yes (random.shuffle)</i>	Yes	Helps prevent the model from learning class patterns based on a fixed order, promoting more balanced and effective training.
Held-out Test Set (separate from training)	<i>Yes / No</i>	<i>No (validation_split only)</i>	No	
Validation Split (of training set)	<i>0.0 - 0.3</i>	<i>0.1</i>	0.2	Provides a suitable validation portion for evaluating the model's generalization and monitoring its performance during training.

3. Model Architecture Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Conv2D Layer 1 – Filters	<i>16 / 32 / 64 / 128</i>	<i>64</i>	32	Using 32 filters provides sufficient capacity to capture important visual features such as edges, textures, and shapes while maintaining a practical balance between model performance and memory usage.
Conv2D Layer 1 – Kernel Size	<i>(3,3) / (5,5)</i>	<i>(3,3)</i>	(3,3)	A 3×3 kernel is a compact and commonly used choice for capturing local image features such as edges, textures, and small patterns.

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Conv2D Layer 1 – Activation	<i>relu / tanh / elu / leaky_relu</i>	<i>relu</i>	<i>relu</i>	ReLU enables efficient gradient-based learning and provides a proven activation function that has already been successfully applied in the notebook.
Pooling Layer 1 – Type & Pool Size	<i>MaxPooling2D / AveragePooling2D; (2,2) / (3,3)</i>	<i>MaxPooling2D, (2,2)</i>	MaxPooling2D, (2,2)	Reduces the spatial dimensions of feature maps while preserving the most important learned features.
Conv2D Layer 2 – Filters	<i>16 / 32 / 64 / 128</i>	<i>64</i>	64	Using 64 filters in the second convolutional layer increases feature-learning capacity for more complex patterns while maintaining a reasonable balance between model performance and memory usage.
Conv2D Layer 2 – Kernel Size	<i>(3,3) / (5,5)</i>	<i>(3,3)</i>	(3,3)	A 3×3 kernel is a compact and commonly used choice for capturing local image features such as edges, textures, and small patterns.
Conv2D Layer 2 – Activation	<i>relu / tanh / elu</i>	<i>relu</i>	<i>relu</i>	ReLU enables efficient gradient-based learning and provides a proven activation function that has already been successfully applied in the notebook.
Pooling Layer 2 – Type & Pool Size	<i>MaxPooling2D / AveragePooling2D; (2,2) / (3,3)</i>	<i>MaxPooling2D, (2,2)</i>	MaxPooling2D, (2,2)	Reduces the spatial dimensions of feature maps while preserving the most important learned features.
Dense (Fully Connected) Units	<i>16 / 32 / 64 / 128</i>	<i>64</i>	64	Using 64 dense units provides sufficient capacity to combine and learn higher-level features extracted by the convolutional layers while maintaining a reasonable balance between classification performance and memory usage.
Dropout Rate	<i>0.0 - 0.5</i>	<i>None (not used)</i>	0.2	A 0.2 dropout rate provides mild regularization to help reduce overfitting while retaining enough model capacity to learn important features effectively.

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Output Layer Activation	<i>sigmoid (binary) / softmax (multi-class)</i>	<i>sigmoid</i>	<i>sigmoid</i>	Binary dog-versus-cat classification uses a single sigmoid output to produce a probability for the two possible classes.

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Loss Function	<i>binary_crossentropy / categorical_crossentropy</i>	<i>binary_crossentropy</i>	<i>binary_crossentropy</i>	Suitable for binary classification tasks where the model uses a sigmoid output to distinguish between two classes.
Optimizer	<i>adam / sgd / rmsprop</i>	<i>adam</i>	<i>adam</i>	Adjusts learning rates adaptively and offers a reliable baseline for training CNN models in image classification tasks.
Learning Rate	<i>0.1 / 0.01 / 0.001 / 0.0001</i>	<i>0.001 (Adam default)</i>	<i>0.001</i>	A commonly used and stable initial learning rate for Adam, providing a practical starting point for effective CNN training.
Batch Size	<i>8 / 16 / 32 / 64 / 128</i>	<i>32</i>	<i>32</i>	Smaller batch sizes allow more frequent weight updates and can provide effective training for image classification tasks while managing memory usage.
Number of Epochs	<i>e.g. 10 - 200</i>	<i>200</i>	<i>30</i>	Using more than the original 5 epochs gives the deeper model sufficient time to learn and converge, while early stopping can prevent unnecessary training when validation accuracy no longer improves.
Early Stopping	<i>Yes (patience = __) / No</i>	<i>No</i>	<i>No</i>	Early stopping was not used to allow the model to complete the planned training epochs and fully learn the available image features without prematurely stopping the training process.

5. Evaluation Results (after training with chosen values)

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	0.9891	$\geq 90\%$ target
Validation Accuracy	0.8524	$\geq 90\%$ target
Test Accuracy	0.9716	$\geq 90\%$ target
Test Loss	0.1899	As low as practical; evaluate together with test accuracy
Precision / Recall / F1 (macro avg)	0.9716 / 0.9716 / 0.9716	Preferably ≥ 0.90 macro F1

5. a. Paste here all screenshots/figures of the tuned model

```
[33]: # =====  
# TRIAL 1 - CNN MODEL  
# =====  
  
X = X / 255.0  
  
# =====  
# ===== CHANGE THESE PARAMETERS FOR FUTURE TRIALS =====  
# =====  
  
# ===== IMAGE PARAMETERS =====  
IMG_HEIGHT = 100      # CHANGE HERE: Image height  
IMG_WIDTH = 100       # CHANGE HERE: Image width  
COLOR_CHANNELS = 3     # CHANGE HERE: RGB = 3, Grayscale = 1  
  
# ===== CONVOLUTION FILTERS =====  
FILTERS_1 = 32        # CHANGE HERE: First Conv2D filters  
FILTERS_2 = 64        # CHANGE HERE: Second Conv2D filters  
FILTERS_3 = 64        # CHANGE HERE: Third Conv2D filters  
  
# ===== KERNEL =====  
KERNEL_SIZE = (3, 3)  # CHANGE HERE: Kernel size  
  
# ===== DENSE LAYER =====  
DENSE_UNITS = 64      # CHANGE HERE: Dense neurons  
  
# ===== DROPOUT =====  
DROPOUT_RATE = 0.2    # CHANGE HERE: Dropout rate
```

```
# ===== DENSE LAYER =====
DENSE_UNITS = 64      # CHANGE HERE: Dense neurons

# ===== DROPOUT =====
DROPOUT_RATE = 0.2    # CHANGE HERE: Dropout rate

# ===== LEARNING RATE =====
LEARNING_RATE = 0.001 # CHANGE HERE: Learning rate

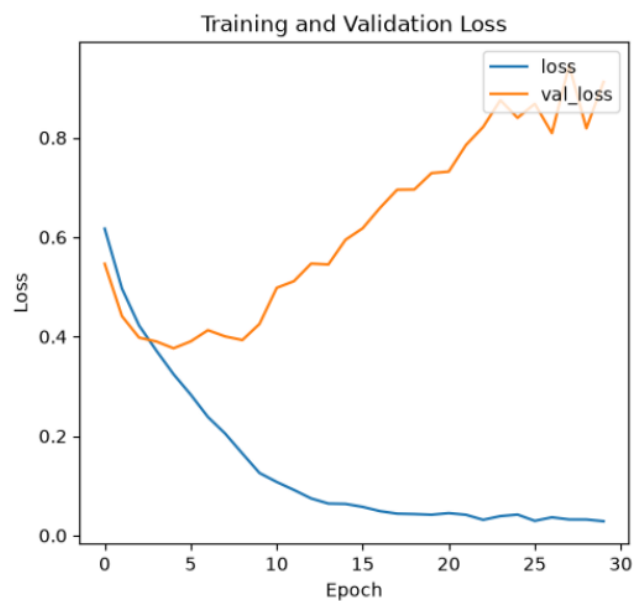
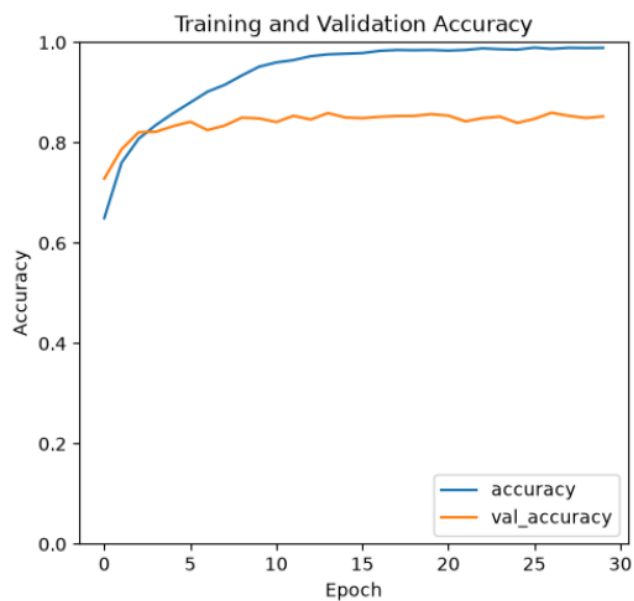
# ===== TRAINING PARAMETERS =====
BATCH_SIZE = 32        # CHANGE HERE: Batch size
EPOCHS = 30            # CHANGE HERE: Number of epochs
VALIDATION_SPLIT = 0.20 # CHANGE HERE: Validation percentage
```

CNN RESULTS

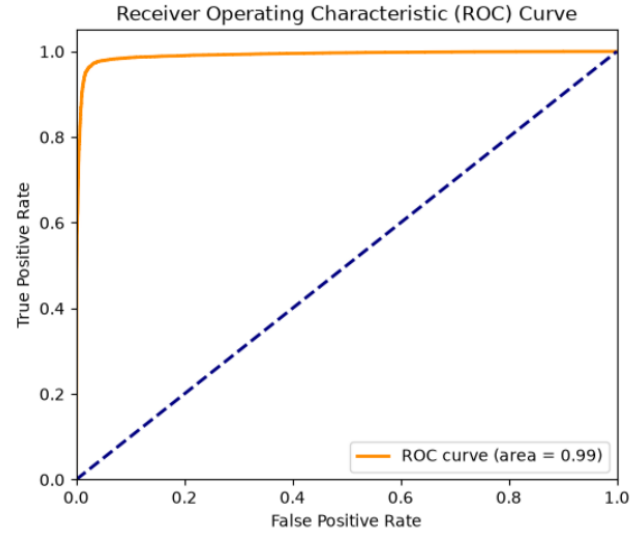
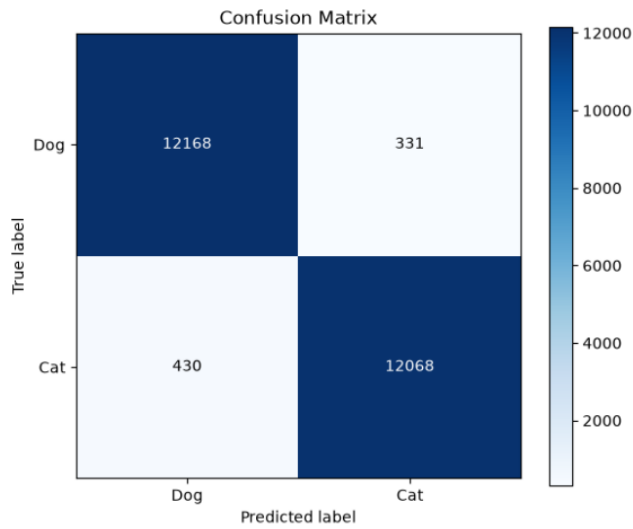
```
Training Accuracy   : 0.9891 (98.91%)
Validation Accuracy : 0.8524 (85.24%)
```

```
Test Accuracy      : 0.9716 (97.16%)
Test Loss          : 0.1899
```

```
Precision (Macro Avg): 0.9716
Recall (Macro Avg)   : 0.9716
F1-Score (Macro Avg) : 0.9716
```



Confusion Matrix:
[[12168 331]
[430 12068]]



6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

The hyperparameters were chosen to ensure sufficient representation of the image's characteristics, the model's performance, and the amount of RAM memory available. Larger image size, using RGB input data, 32 and 64 convolutional filters, and 64 dense units allow the CNN to adequately learn the characteristics of dogs and cats. The dropout value of 0.2 was selected to prevent the model from overfitting, while early stopping was not used to avoid stopping the training too early. The batch size and images settings were also selected with RAM memory in mind. Thus, the chosen hyperparameters should ensure sufficient stability and convergence of the model during training while preventing overfitting and using the available memory efficiently.